

INTRODUCTION TO DATA STRUCTURE

Unit 1

Presented By: Saroj Bhandari

Syllabus

- 1.1 Preliminary data type and Abstract data type**
- 1.2 Data structure, Need and Types of Data Structure**
- 1.3 Comparison between ADT and Data Structure**
- 1.4 Review of Array, Structure and Pointer**

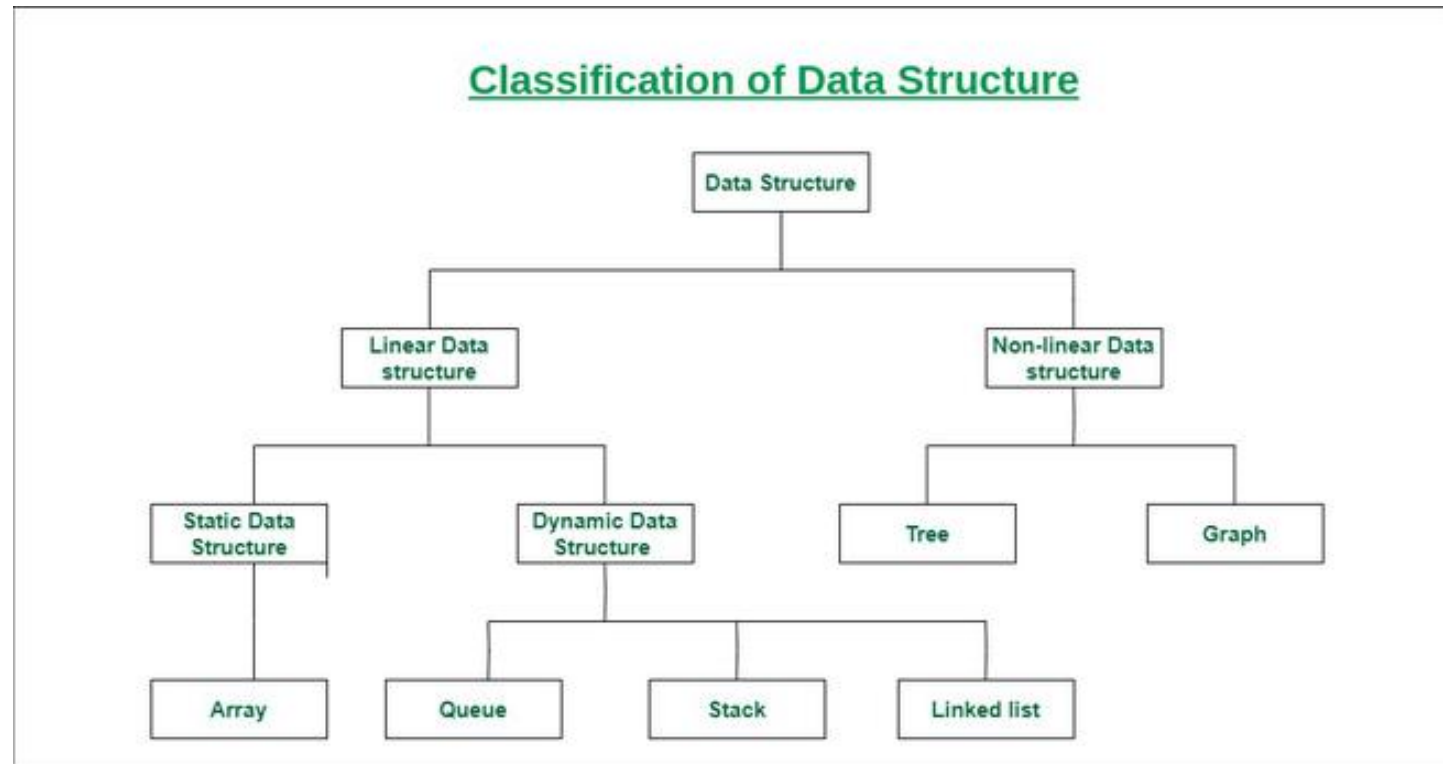
Data Structure

- A **data structure** is a particular way of organizing data in a computer so that it can be used effectively. The idea is to reduce the space and time complexities of different tasks.
- The choice of a good data structure makes it possible to perform a variety of **critical operations effectively**.
- An efficient data structure also uses minimum memory space and execution time to process the structure.
- A data structure is not only used for organising the data. It is also used for **processing, retrieving, and storing data**.
- There are different basic and advanced types of data structures that are used in almost every program or software system that has been developed. So we must have good knowledge of **data structures**.

Need Of Data Structure:

- The structure of the data and the synthesis of the algorithm are relative to each other.
- Data presentation must be easy to understand so the developer, as well as the user, can make an efficient implementation of the operation.
- Data structures provide an easy way of **organizing, retrieving, managing, and storing data.**
- **Here is a list of the needs for data.**
 - Data structure modification is easy.
 - It requires less time.
 - Save storage memory space.
 - Data representation is easy.
 - Easy access to the large database

Classification/Types of Data Structures:



Types of Data Structures

- **Linear Data Structure:**

- Elements are arranged in one dimension ,also known as linear dimension.
- Example: lists, stack, queue, etc.

- **Non-Linear Data Structure**

- Elements are arranged in one-many, many-one and many-many dimensions.
- Example: tree, graph, table, etc.

Applications of Data Structures:

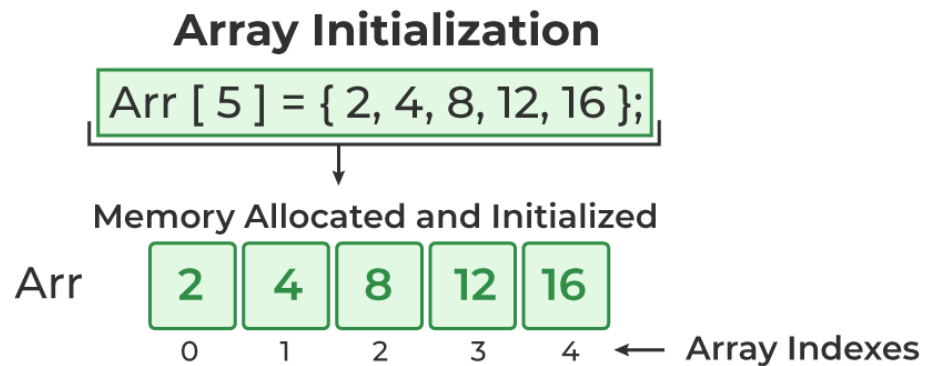
Data structures are used in various fields such as:

- Operating system
- Graphics
- Computer Design
- Blockchain
- Genetics
- Image Processing
- Simulation

Let's Explore type of Data Structure:

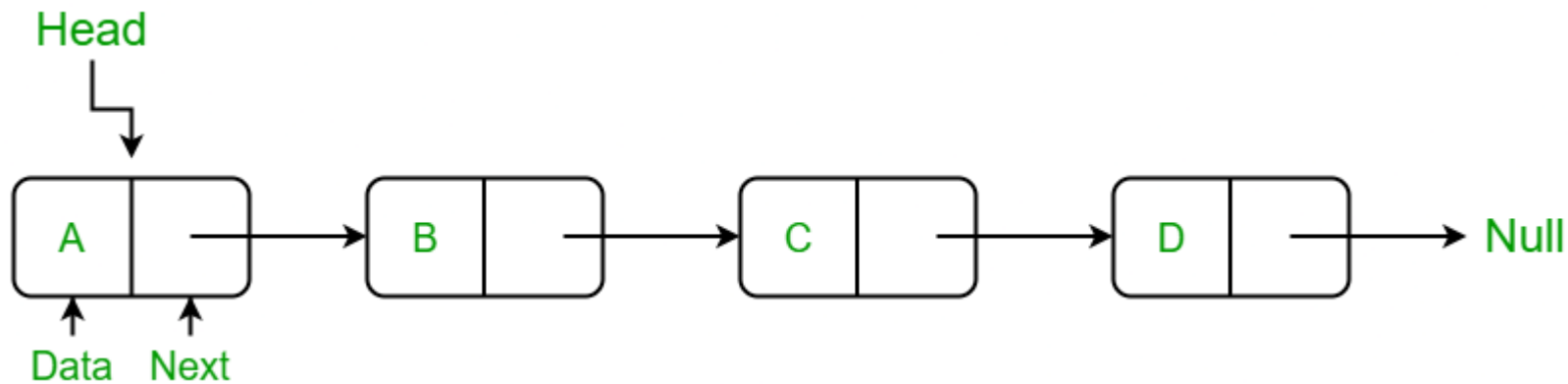
1. Array:

- An array is a collection of data items stored at contiguous memory locations.



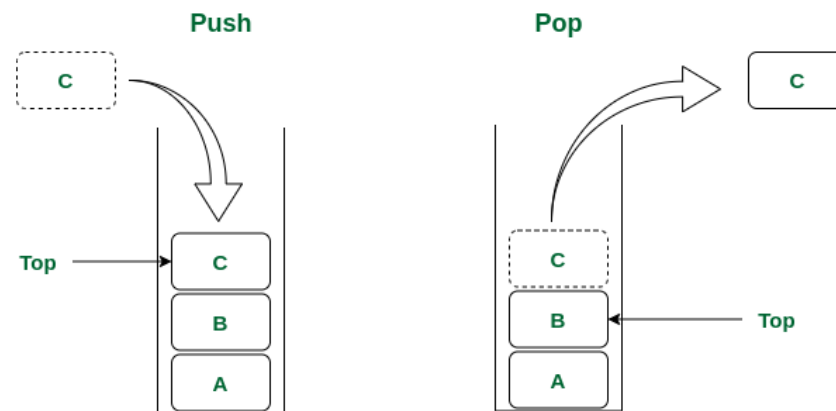
2. Linked Lists:

- Like arrays, **Linked List** is a linear data structure. Unlike arrays, linked list elements are not stored at a contiguous location; the elements are linked using pointers.



3. Stack:

- Stack is a **linear data structure** which follows a particular order in which the operations are performed.
- The order may be **LIFO(Last In First Out)** or **FILO(First In Last Out)**. In stack, all insertion and deletion are permitted at only **one end of the list**.



Stack Data Structure

4. Queue:

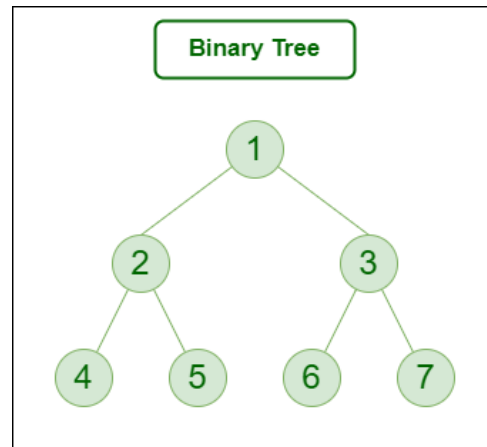
- Like Stack, Queue is a linear structure which follows a particular order in which the operations are performed. The order is First In First Out (FIFO). In the queue, items are inserted at one end and deleted from the other end.



Queue Data Structure

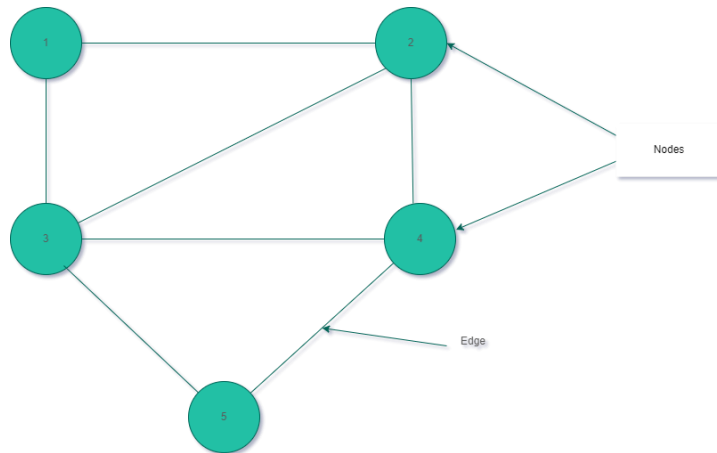
5. Binary Tree:

- Unlike Arrays, Linked Lists, Stack and queues, which are linear data structures, trees are hierarchical data structures.
- A **binary tree** is a tree data structure in which each node has at most two children, which are referred to as the left child and the right child. It is implemented mainly using Links.



6. Graph

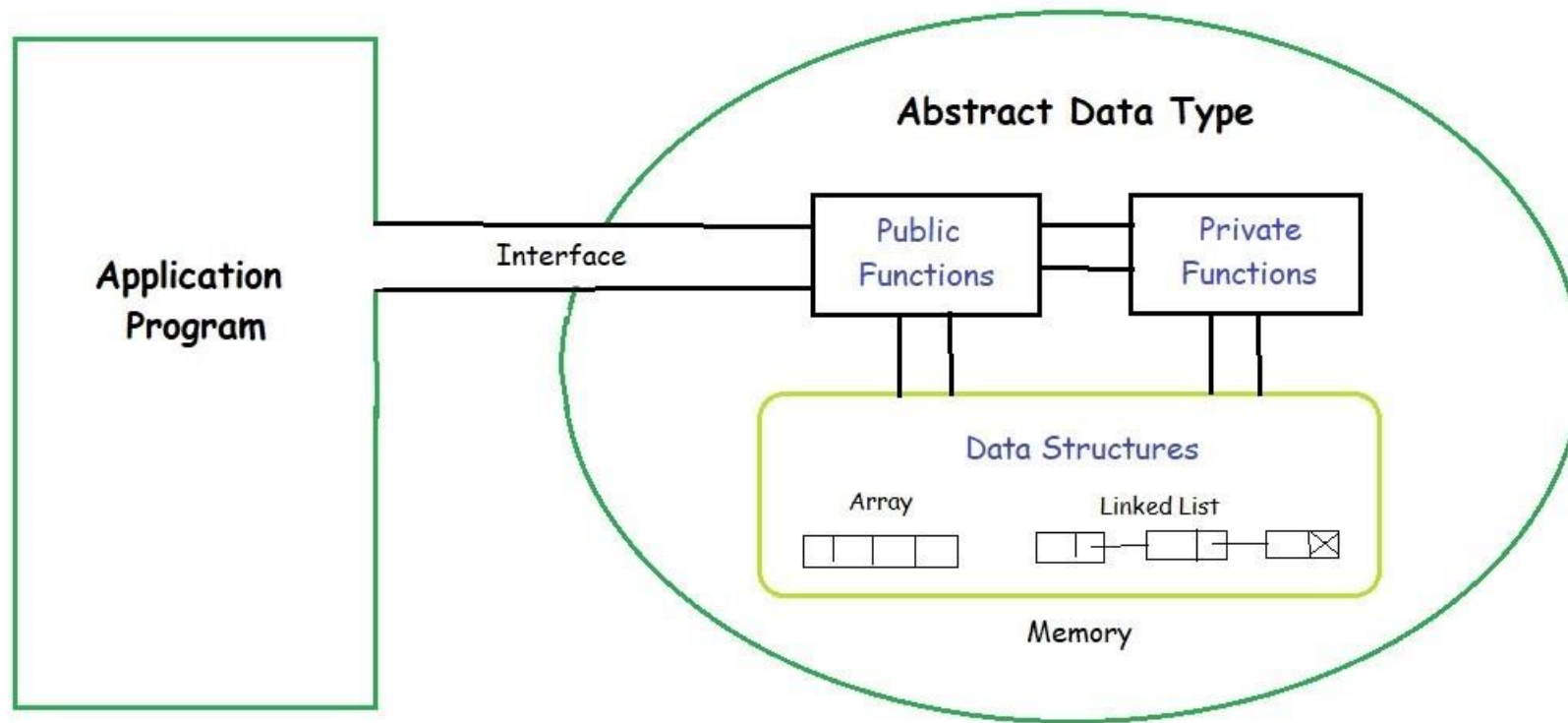
- Graph is a data structure that consists of a collection of **nodes (vertices)** **connected by edges**.
- Graphs are used to represent relationships between objects and are widely used in computer science, mathematics, and other fields.



Abstract Data Type(ADT)

- An **Abstract Data Type (ADT)** is a conceptual model that defines a set of operations and behaviors for a data structure, without specifying how these operations are implemented or how data is organized in memory.
- The definition of ADT only mentions what operations are to be performed but not how these operations will be implemented.
- It does not specify **how data will be organized in memory and what algorithms will be used for implementing the operations.**
- It is called “abstract” because it provides an **implementation-independent view.**
- The process of providing only the essentials and hiding the details is known as **abstraction**

Abstract Data Types



Preliminary Datatype VS Abstract Datatype

Feature	Preliminary Datatype	Abstract Datatype (ADT)
Definition	Basic or built-in data types provided by the language	User-defined types that encapsulate data and operations
Examples	int, float, char, bool	Stack, Queue, Linked List, Tree, Graph
Abstraction Level	Low (closer to machine level)	High (closer to real-world modeling)
Data + Operations	Only stores data	Stores data and defines operations on that data
Implementation	Directly supported and implemented by the language	Implementation is hidden; only interface is exposed
Modifiability	Cannot change internal behavior	You can define your own operations and behavior
Use Case	Used for simple data storage and manipulation	Used for solving complex problems with data structure logic
Encapsulation	No	Yes (data + operations bundled together)
Customizability	Limited	Highly customizable

Comparison between **ADT** and **Data Structure**

Aspect	Abstract Data Type (ADT)	Data Structure
Concept vs Implementation	ADT is a conceptual model describing data and operations	Data structure is the actual implementation of an ADT in memory
Level of Abstraction	High-level: focuses on what operations are possible	Low-level: focuses on how operations are carried out
Interface vs Internal Details	Defines interface only (e.g., push, pop, insert)	Provides internal structure and logic to perform those operations
Language Independence	Independent of any programming language	Depends on programming language and environment
Purpose	To provide a blueprint for organizing data and defining behavior	To efficiently store, access, and modify data in programs

Review of **Array**, Structure and Pointers

Topic	Descriptive Review
Array	An array is a linear data structure that stores a fixed-size sequence of elements of the same data type in contiguous memory locations. It allows accessing elements using index numbers starting from 0. Arrays are ideal for storing and processing large collections of data efficiently when the size is known beforehand. <pre>int numbers[5] = {10, 20, 30, 40, 50}; printf("%d", numbers[2]); // Output: 30</pre>

Review of Array, **Structure** and Pointers

- A structure is a user-defined data type that groups variables of different types into a single unit.
- This allows you to model real-world entities by bundling related information together, such as a student's ID, name, and marks, under one variable.
- Structures help organize complex data more meaningfully and are fundamental for creating more advanced data models in C and C++.

```
int a = 10;
```

```
int *ptr = &a;
```

```
printf("%d", *ptr); // Output: 10
```

Review of Array, **Structure** and Pointers

```
struct Student {  
  
    int id;  
  
    char name[30];  
  
    float marks;  
  
};  
  
struct Student s1 = {1, "Alice", 89.5};  
  
printf("Name: %s", s1.name); // Output: Alice
```

Review of Array, Structure and **Pointers**

- A **pointer** is a special variable that stores the memory address of another variable. Pointers enable indirect access to data and are crucial for dynamic memory management, efficient array handling, and implementing complex data structures like linked lists.
- Mastering pointers is essential for low-level programming and optimizing performance in C and C++ applications.

```
int a = 10;
```

```
int *ptr = &a;
```

```
printf("%d", *ptr); // Output: 10
```

Thank You !!!!!